DATA STRUCTURES CSLR31

Course Project Report

CampusBuggyDispatch

Campus Electric-Buggy Routing and Dispatch System

Submitted by

Team Debuggers

Krishnendu

Tharuni

Vishak

Department of Computer Science and Engineering

National Institute of Technology Tiruchirappalli

2026

Declaration

We hereby declare that the project report entitled *CampusBuggyDispatch: Campus Electric-Buggy Routing and Dispatch System* is a record of the work carried out by our team as part of the Data Structures CSLR31 course project. The report describes the design and implementation represented in the project repository. Any external resources used are acknowledged in the references.

Team Debuggers
Krishnendu Tharuni Vishak

Acknowledgement

We express our sincere gratitude to the course instructor and teaching assistants for their guidance in data structures, graph algorithms and modular programming. We also acknowledge the developers of the cJSON library, which is bundled with the project and used for JSON parsing and generation.

Abstract

CampusBuggyDispatch is a modular, C-based simulation of an electric-buggy dispatch system for a campus environment. The system models campus locations as vertices in a weighted graph and the roads connecting them as edges. It accepts passenger requests and buggy-fleet information from JSON files, calculates routes using Dijkstra's shortest-path algorithm, and assigns eligible buggies according to configurable dispatch strategies. The project implements four dispatch modes: `shortest`, `greedy`, `regret` and `smart`. Request prioritisation can incorporate both urgency and waiting-time ageing, while fleet assignment observes vehicle availability and passenger capacity. The simulation advances in discrete time steps and exports request outcomes and summary metrics in JSON or CSV format. This project demonstrates the application of graph representation, shortest-path algorithms, scheduling heuristics, modular C programming and file-based data processing to a campus mobility problem. The current implementation is a static-data command-line simulation; it does not provide live GPS tracking, real-time traffic updates or a passenger-facing booking application.

Keywords: Graphs, Dijkstra's algorithm, dispatch scheduling, priority ageing, C programming, campus mobility, simulation.

Contents

1 Introduction **1**

1.1 Background . . . 1

1.2 Problem Statement . . . 1

1.3 Project Objectives . . . 2

1.4 Scope . . . 2

2 Problem Relevance and Sustainable Development Goals **3**

2.1 Campus Mobility Context . . . 3

2.2 Relevance to the United Nations Sustainable Development Goals . . . 3

3 System Requirements and Input Model **5**

3.1 Functional Requirements . . . 5

3.2 Non-functional Considerations . . . 5

3.3 Input Files . . . 5

3.3.1 Campus Nodes . . . 6

3.3.2 Campus Edges . . . 6

3.3.3 Buggy Records . . . 6

3.3.4 Passenger Requests . . . 7

3.3.5 Simulation Configuration . . . 7

4 Methodology and Algorithms **9**

4.1 Graph Representation . . . 9

4.2 Shortest-Path Routing . . . 9

4.3 Request Priority and Ageing . . . 9

4.4 Dispatch Strategies . . . 10

4.5 Eligibility Constraints . . . 10

4.6 Discrete-Time Simulation . . . 10

5 System Design and Architecture **12**

5.1 Overall Architecture . . . 12

5.2 Execution Workflow . . . 12

5.3 Module Responsibilities 13

6 Implementation Details **15**

6.1 Modular C Programming 15

6.2 Memory and Data Handling 15

6.3 Output Generation 15

7 Build, Execution and Testing **16**

7.1 Build Requirements 16

7.2 Compilation 16

7.3 Running the Simulation 16

7.4 Testing 17

7.5 Test Scenarios 17

7.6 Evaluation Metrics 17

8 Limitations and Future Enhancements **19**

8.1 Current Limitations 19

8.2 Future Enhancements 19

9 Conclusion **21**

Team Contributions **22**

References **23**

List of Figures

5.1 High-level architecture of CampusBuggyDispatch 12
5.2 Simulation workflow 13

List of Tables

3.1 Simulation configuration fields 8

5.1 Main source files and responsibilities 13

7.1 Simulation output metrics 18

9.1 Team contribution record 22

Chapter 1

Introduction

1.1 Background

Large campuses often contain academic blocks, hostels, laboratories, administrative buildings and other facilities spread across a wide area. Walking between distant locations may be inconvenient, and a limited fleet of shared electric buggies may be used to support campus transportation. When several passenger requests arrive close together, assigning vehicles manually can lead to unnecessary waiting, inefficient vehicle movement and poor utilisation of the available fleet.

CampusBuggyDispatch explores how data structures and algorithms can support a more systematic dispatch process. The project represents the campus as a graph, calculates routes between locations, checks buggy eligibility and selects assignments using configurable policies.

1.2 Problem Statement

Given a campus road network, a fleet of electric buggies and a set of passenger requests, the system must determine feasible buggy-request assignments while considering:

- the route distance and estimated travel time from a buggy's current location to a pickup point;
- the passenger capacity and availability status of each buggy;
- the destination and estimated passenger-trip duration;
- the urgency and waiting time of each request; and
- the maximum duration of the simulation.

The objective is to simulate dispatch decisions and report service outcomes in a consistent, measurable format.

1.3 Project Objectives

The main objectives are:

1. Represent campus locations and roads using a graph data structure.
2. Implement shortest-path routing using Dijkstra's algorithm.
3. Model buggy availability, capacity and location.
4. Represent passenger requests with pickup, destination, passenger count, urgency and arrival time.
5. Implement and compare multiple dispatch strategies.
6. Incorporate waiting-time ageing into request priority.
7. Export simulation outcomes in JSON or CSV format.
8. Organise the implementation into reusable C modules with header and source files.

1.4 Scope

The submitted project is a command-line simulation driven by static JSON input files. Road distances and average travel times are supplied as input data. Buggy locations are updated by the simulation itself. The implementation does not connect to GPS hardware, live traffic services, a mobile booking application or a network API.

Chapter 2

Problem Relevance and Sustainable Development Goals

2.1 Campus Mobility Context

A limited shared-transport fleet is a resource-allocation problem. The dispatch system must choose which request to serve and which vehicle to assign, while ensuring that the selected vehicle can reach the pickup point and accommodate the requested number of passengers. Graph algorithms provide a way to reason about route costs, while scheduling heuristics help prioritise competing requests.

2.2 Relevance to the United Nations Sustainable Development Goals

The project has conceptual relevance to the following Sustainable Development Goals (SDGs):

- **SDG 11 – Sustainable Cities and Communities:** The project explores more organised shared mobility within a campus, which is a small-scale example of improving the accessibility and efficiency of transport systems.
- **SDG 9 – Industry, Innovation and Infrastructure:** The implementation applies computational methods and software engineering to a practical transportation-infrastructure problem.
- **SDG 13 – Climate Action:** Electric-buggy transport can form part of lower-emission mobility options. The simulation itself does not measure emissions or establish a quantified environmental impact.

These connections describe the project's motivation and potential application. They

should not be interpreted as measured SDG outcomes.

Chapter 3

System Requirements and Input Model

3.1 Functional Requirements

The system is designed to:

1. load the campus graph, buggy fleet, passenger requests and simulation configuration;
2. calculate routes between graph nodes;
3. identify buggies that are available and have sufficient capacity;
4. evaluate requests that have arrived by the current simulation time;
5. select assignments according to the configured dispatch policy;
6. update vehicle state as trips progress and complete; and
7. write summary and per-request results to a file.

3.2 Non-functional Considerations

The code is organised into separate modules for graph routing, dispatch and output. The program uses JSON files for input and supports repeatable scenarios. The simulation can be configured through a time step, a maximum simulation duration, dispatch-policy selection and output format.

3.3 Input Files

The program expects the following five files in the selected data directory:

- `simulation_config.json`
- `campus_nodes.json`

- `campus_edges.json`
- `campus_buggies.json`
- `requests.json`

The default data is stored in `Data/`. Alternative complete scenarios are provided under `Data/scenarios/aging/` and `Data/scenarios/expanded/`.

3.3.1 Campus Nodes

Each campus node contains a unique integer identifier, a name and geographic coordinates. Requests and buggy records refer to nodes using their IDs.

Listing 3.1: Illustrative campus node record

```
{
  "id": 1,
  "name": "Orion",
  "lat": 10.7597475,
  "lon": 78.8107876
}
```

3.3.2 Campus Edges

Each edge specifies its source and destination node, distance in metres, average travel time in seconds and whether the connection is bidirectional.

Listing 3.2: Illustrative campus edge record

```
{
  "from": 1,
  "to": 2,
  "distance_m": 494,
  "avg_time_s": 380,
  "bidirectional": true
}
```

3.3.3 Buggy Records

A buggy record contains an identifier, passenger capacity, current location, availability status and speed.

Listing 3.3: Illustrative buggy record

```
{
```

```
  "id": "BG1",
  "capacity": 6,
  "current_location": 1,
  "status": "AVAILABLE",
  "speed_mps": 5.5
}
```

3.3.4 Passenger Requests

A request contains an identifier, pickup node, destination node, passenger count, urgency and request arrival time.

Listing 3.4: Illustrative passenger request

```
{
  "id": "R1",
  "pickup": 22,
  "destination": 21,
  "passengers": 1,
  "urgency": 2,
  "request_time": 13
}
```

Urgency is supplied as a numeric input value. The current project does not infer urgency automatically from a passenger category or destination.

3.3.5 Simulation Configuration

The configuration controls the dispatch policy, routing label, priority weights, simulation time step, maximum simulation duration and output format.

Listing 3.5: Illustrative simulation configuration

```
{
  "algorithm": "shortest",
  "routing": "dijkstra",
  "aging_weight": 0.5,
  "urgency_weight": 1.0,
  "time_step_s": 10,
  "max_simulation_time_s": 1800,
  "output_format": "csv"
}
```

Table 3.1: Simulation configuration fields

Field	Meaning
`algorithm`	Dispatch policy: shortest, greedy, regret or smart.
`routing`	Routing label; the implemented route calculation is Dijkstra.
`aging_weight`	Weight applied to elapsed waiting time.
`urgency_weight`	Weight applied to request urgency.
`time_step_s`	Duration of one simulation tick, in seconds.
`max_simulation_time_s`	Maximum simulation duration, in seconds.
`output_format`	Output format: json or csv.

Chapter 4

Methodology and Algorithms

4.1 Graph Representation

The campus is represented as a weighted graph $G = (V, E)$, where V is the set of campus locations and E is the set of road connections. The implementation uses an adjacency-list representation. Each edge stores its distance and average travel time. A bidirectional edge is made available in both directions; a one-way edge is available only in its specified direction.

4.2 Shortest-Path Routing

Dijkstra's algorithm is used to find a minimum-distance route between a source node and a destination node. The edge distance is the route weight. The route result includes the sequence of node IDs, total distance and associated travel time accumulated along the selected route. Dijkstra's algorithm is appropriate because the supplied road distances are non-negative.

For a graph represented with an adjacency list and a binary min-priority queue, the commonly used time complexity is $O((|V|+|E|)\log|V|)$. The exact implementation-level complexity depends on the data structure used for selecting the next minimum-distance vertex.

4.3 Request Priority and Ageing

The request priority is based on urgency and elapsed waiting time:

$$P(r) = w_u U(r) + w_a \max(0, t_{\text{now}} - t_{\text{request}}), \tag{4.1}$$

where:

- $P(r)$ is the priority of request r;
- $U(r)$ is the urgency value supplied for the request;
- t_{now} is the current simulation time;
- t_{request} is the request's arrival time;
- w_u is the urgency weight; and
- w_a is the ageing weight.

The ageing term increases as a request waits, allowing older requests to gain priority over time. The weights are configurable through the simulation configuration file.

4.4 Dispatch Strategies

The project supports four dispatch modes:

`shortest`
Evaluates feasible request–buggy pairs and selects the pair with the lowest combined pickup ETA and passenger-trip duration.

`greedy`
Selects the request with the highest urgency-and-age priority and uses its nearest eligible buggy.

`regret`
Adds a regret component based on the difference between the nearest and second-nearest eligible buggy ETAs to the request priority.

`smart`
Currently follows the same regret-based selection mode as `regret`.

4.5 Eligibility Constraints

A buggy is eligible for a request only when it is available and its passenger capacity is at least the number of passengers in the request. Route reachability is also required. A buggy marked `NOT_AVAILABLE` in the input remains offline during the simulation.

4.6 Discrete-Time Simulation

The simulation advances in fixed increments defined by `time_step_s`. At each tick, the system:

1. releases buggies whose assigned trips have completed and updates their locations;
2. considers requests whose arrival time has been reached;
3. checks buggy availability, capacity and route reachability;
4. calculates route estimates and applies the configured dispatch policy;
5. records assignments and updates vehicle state; and
6. continues until the configured simulation horizon is reached.

A request is not assigned if the pickup and passenger trip cannot finish within the configured simulation horizon.

Chapter 5

System Design and Architecture

5.1 Overall Architecture

The program is divided into a simulation coordinator, graph-routing module, dispatch module and output module. The bundled cJSON library supports JSON input and output.

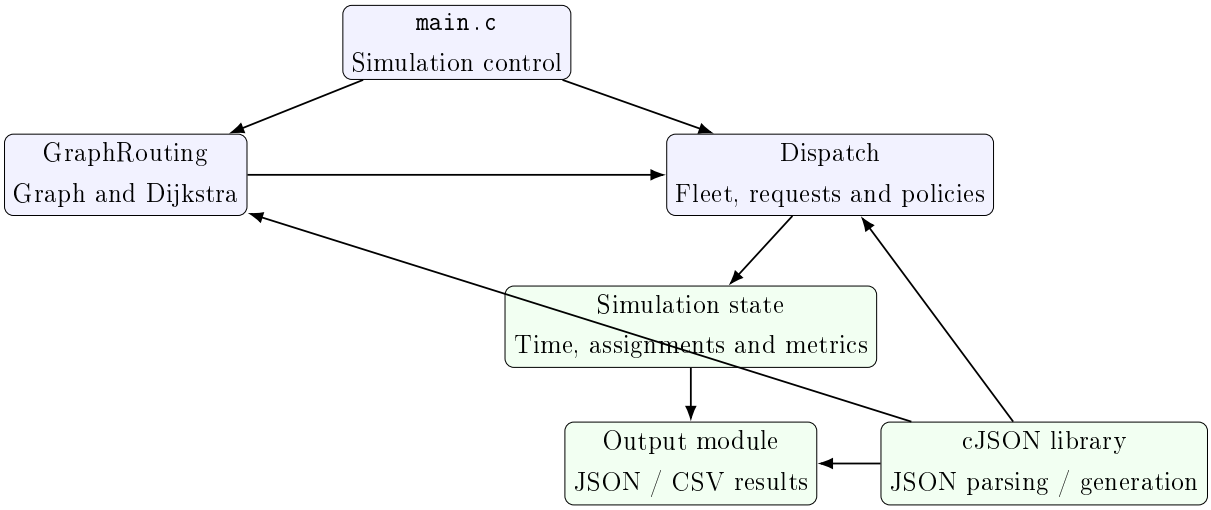


Figure 5.1: High-level architecture of CampusBuggyDispatch

5.2 Execution Workflow

The overall execution flow is illustrated in Figure 5.2.

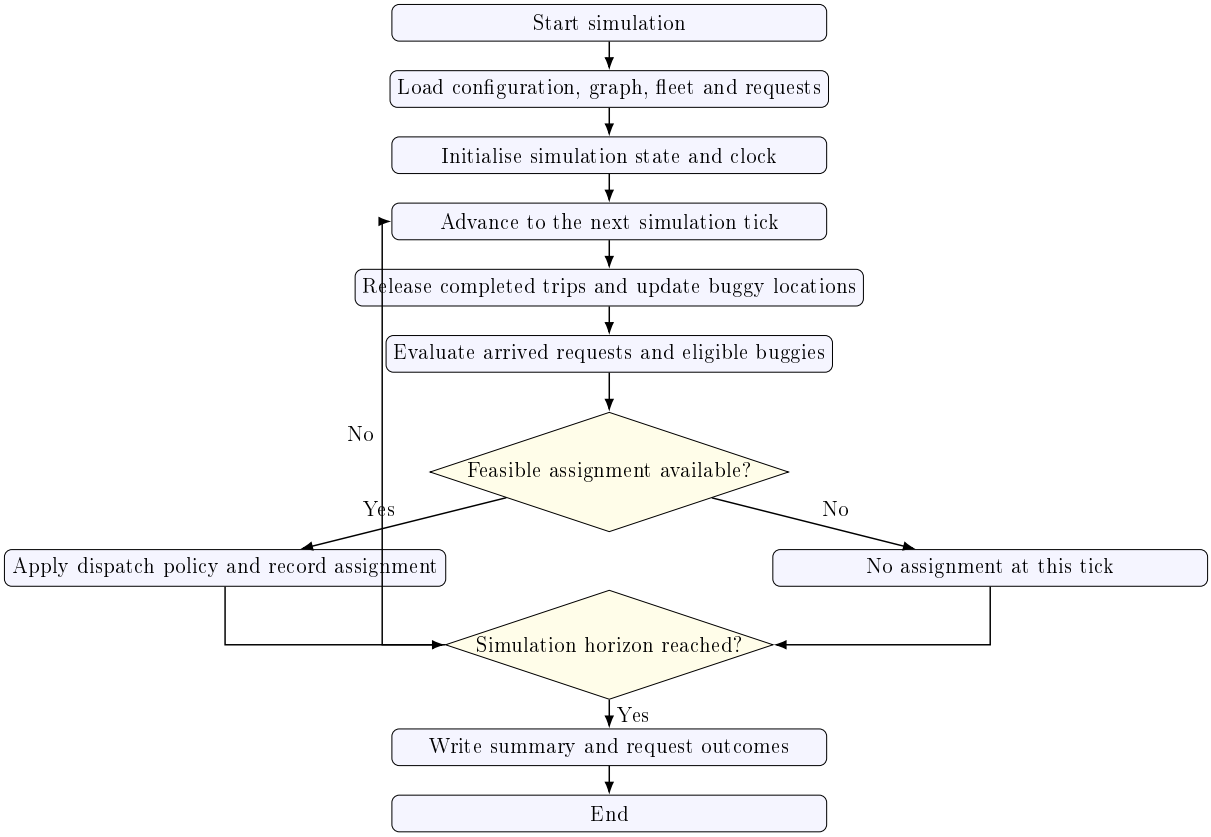


Figure 5.2: Simulation workflow

5.3 Module Responsibilities

Table 5.1: Main source files and responsibilities

File	Responsibility
`main.c`	Entry point and simulation coordinator. Loads configuration and data, advances simulation time, processes completed trips, invokes dispatch and records outcomes.
`GraphRouting/graph.h`	Declares graph structures such as `Node`, `Edge`, `AdjNode`, `Graph` and `Route`, along with public graph functions.
`GraphRouting/graph.c`	Implements graph loading, adjacency-list construction and Dijkstra's shortest-path calculation.
`Dispatch/dispatch.h`	Declares `Buggy`, `Request` and `AssignmentResult`, together with dispatch-related interfaces.
`Dispatch/dispatch.c`	Loads buggy and request data, computes priority, filters eligible buggies and applies dispatch policies.
`Output/outputwriter.h`	Declares `RequestOutcome`, `Metrics` and output function interfaces.

File	Responsibility
`Output/outputwriter.c`	Writes simulation summaries and per-request outcomes in JSON or CSV format.
`lib/cJSON.h`, `lib/cJSON.c`	Provide JSON parsing and generation support.
`tests/test_all.c`	Integration test program for graph, dispatch and priority behaviours.

Chapter 6

Implementation Details

6.1 Modular C Programming

The implementation separates declarations from definitions. Header files expose shared structures and function declarations, while corresponding C source files contain implementations. This separation helps keep graph processing, dispatch decisions and result generation independent. The `main.c` file coordinates the modules rather than containing all the routing and dispatch logic.

6.2 Memory and Data Handling

The graph and simulation data are loaded from JSON files using the bundled cJSON library. The graph module constructs adjacency lists from the loaded road records. The program also provides graph and route cleanup interfaces such as `free_graph` and `free_route`. Dynamic data structures must be released after use to avoid memory leaks.

6.3 Output Generation

The output module supports two formats:

- **JSON:** writes structured simulation summary and request-level results to `results.json`.
- **CSV:** writes tabular simulation results to `results.csv`.

The output includes total, served and unserved request counts, average waiting time and average buggy utilisation. Per-request information includes assignment details, waiting time and pickup ETA.

Chapter 7

Build, Execution and Testing

7.1 Build Requirements

The project requires:

- a C compiler such as GCC or Clang;
- CMake version 3.15 or later; and
- a terminal or command prompt.

The cJSON source files are included under `lib/`; a separate cJSON installation is not required.

7.2 Compilation

From the repository root, run:

Listing 7.1: CMake build commands

```
cmake -S . -B build
cmake --build build
```

The build defines the main executable target `buggy_dispatch` and the test executable target `test_all`.

7.3 Running the Simulation

The default scenario uses the `Data/` directory:

```
./build/buggy_dispatch
```

The supplied alternative scenarios can be run as follows:

```
./build/buggy_dispatch Data/scenarios/aging
./build/buggy_dispatch Data/scenarios/expanded
```

The selected scenario directory must contain the five required JSON files. The dispatch algorithm and output format are read from that directory's `simulation_config.json`.

7.4 Testing

The CMake project registers the integration test with CTest. It can be run using:

```
ctest --test-dir build --output-on-failure
```

The test executable can also be invoked directly:

```
./build/test_all
```

The integration test source, `tests/test_all.c`, covers behaviours including request priority and ageing, graph loading and shortest-path routing, dispatch assignment, passenger-capacity restrictions and regret-based dispatch behaviour. The repository also contains module-level test source files under `GraphRouting/` and `Dispatch/`; the CMake configuration registers `test_all` as the CTest integration test target.

7.5 Test Scenarios

- **Default scenario:** the campus graph, buggy fleet, requests and configuration under `Data/`.
- **Ageing scenario:** the scenario under `Data/scenarios/aging/`, configured for the greedy strategy and ageing-related weights.
- **Expanded scenario:** the larger scenario under `Data/scenarios/expanded/`, with its own graph, fleet, requests and configuration.

7.6 Evaluation Metrics

The simulation reports metrics intended to support comparison between dispatch policies.

Table 7.1: Simulation output metrics

Metric	Interpretation
`total_requests`	Total number of requests loaded.
`served`	Requests assigned and included in served outcomes.
`unserved`	Requests not served within the simulation horizon.
`avg_wait_time_s`	Average request-to-pickup time for served requests, including pickup ETA.
`avg_buggy_utilization_pct`	Busy buggy time as a percentage of available fleet time within the simulation window.
`wait_time_s`	Request-specific time from request arrival until buggy pickup.
`eta_s`	Estimated time for the assigned buggy to reach the pickup point.

For a fair comparison, the dispatch strategies should be run with the same campus graph, buggy fleet, request set and simulation horizon. A strategy should not be judged using average waiting time alone; served and unserved request counts and buggy utilisation should also be considered. No numerical performance result is asserted in this report because a verified execution output has not been included in the supplied project documentation. Actual results from the program can be inserted here after running each scenario.

Chapter 8

Limitations and Future Enhancements

8.1 Current Limitations

- Input data is loaded from static JSON files; the program does not ingest live passenger requests.
- Buggy positions are updated by the simulation and are not received from GPS hardware.
- Road travel times are static input values and are not updated using live traffic.
- Urgency is supplied numerically and is not inferred automatically from trip purpose or passenger category.
- The project is a command-line simulation and does not include a passenger-facing mobile application, operator dashboard or HTTP API.
- The `smart` dispatch mode currently uses the same regret-based selection mode as `regret`.

8.2 Future Enhancements

Possible extensions include:

1. integrating live GPS updates from campus buggies;
2. adding a passenger interface for submitting pickup and destination requests;
3. developing an operator dashboard for fleet monitoring;
4. updating route travel times using live traffic or measured trip data;
5. deriving urgency from configurable rules, such as accessibility needs or destination categories;

6. exploring ride pooling and multi-stop route optimisation; and
7. evaluating the system using real, anonymised campus mobility data.

Chapter 9

Conclusion

CampusBuggyDispatch demonstrates how graph data structures and scheduling heuristics can be combined to model a campus electric-buggy dispatch problem. The campus road network is represented using an adjacency list, and Dijkstra's algorithm is used to calculate minimum-distance routes. The dispatch module checks vehicle availability and passenger capacity before applying one of four selection modes. Priority ageing provides a mechanism for incorporating waiting time alongside request urgency. A discrete-time simulation coordinates assignments and trip completion, while the output module produces JSON or CSV summaries.

The project brings together several data structures and algorithmic concepts in a modular C implementation. Its current form is a static-data simulation, but the architecture provides a foundation for future work involving live location data, passenger interfaces, improved traffic estimates and more advanced dispatch optimisation.

Team Contributions

The project was developed by Team Debuggers. The individual contributions of the team members are listed below.

Table 9.1: Team contribution record

Team member	**Specific contribution(s)**
Krishnendu	Testing and integration.
Tharuni	Graph routing and implementation of Dijkstra's shortest-path algorithm.
Vishak	Buggy-request allocation using smart dispatch algorithms.

References

1. E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
2. CMake Documentation. Available: https://cmake.org/documentation/
3. GCC Documentation. Available: https://gcc.gnu.org/onlinedocs/
4. Dave Gamble, cJSON repository and documentation. Available: https://github.com/DaveGamble/cJSON
5. United Nations, Sustainable Development Goals. Available: https://sdgs.un.org/goals
6. UNESCO, Sustainable Development Goals. Available: https://www.unesco.org/en/sdgs